

Smart City Traffic Intelligence

From Data Analytics to AI Powered Mobility

Final Capstone Report: Methodology and Findings Across Parts 1 to 3

Gautam K Das

Executive Summary

This report closes out the NUSSOC AI, ML and Data Science capstone, **Smart City Traffic Intelligence**, by pulling together the methodology and findings from all three parts into one document: Part 1's SQL, statistical and Power BI analysis, Part 2's reproducible Python pipeline, and Part 3's machine learning, deployment and responsible AI work. Every part shares the same source, the Metro Interstate Traffic Volume dataset (**48,204 hourly records** of westbound I-94 traffic near Minneapolis and Saint Paul, **2012-10-02 to 2018-09-30**, with weather and US federal holiday data), and each part builds directly on the cleaning and features established by the one before it.

The single most consistent finding across every technique used in this project, SQL aggregation, Power BI charts, statistical correlation, and every machine learning model's own feature importances, is that **time of day and day of week dominate traffic volume, while weather conditions have only a weak secondary effect**. A companion, **equally consistent theme is that the project repeatedly caught and corrected its own subtle errors through direct, skeptical checks rather than accepting first results, from a placeholder temperature reading discovered in Part 1 to a project wide logging bug found and fixed in Part 3**. A separate document, **part3_bias_fairness_governance_report.docx**, covers Task 7's bias, fairness, governance and sustainability discussion in full and is only summarised here to avoid duplication.

Introduction and Dataset

The scenario throughout this capstone is a Smart City Mobility Analytics Team tasked with turning raw traffic sensor data into an end-to-end traffic intelligence solution. The dataset, **data/Metro_Interstate_Traffic_Volume.csv**, contains 48,204 raw hourly rows and 9 columns: holiday, temp, rain_1h, snow_1h, clouds_all, weather_main, weather_description, date_time, and traffic_volume, covering a single corridor and direction over roughly six years. After the cleaning established in Part 1 and formalised in Part 2's pipeline, the working dataset used throughout Parts 2 and 3 is **40,575 rows**, one row per genuinely distinct hour.

No real accident dataset was ever available for this project. Part 3's classification work uses a documented proxy, **high_risk**, built from traffic volume and weather rather than from any observed collision record; this is stated here plainly, as the README and the bias and fairness report both require, and its risks are discussed fully in that separate report.

Part 1: Data Analytics

Part 1 established the dataset's core patterns using SQL, descriptive statistics, probability, and a Power BI dashboard, before any Python code was written. Full working, every query and every calculation, is in **part1_analysis_notes.docx**; this section summarises its methodology and headline findings.

Task 1: SQL Based Traffic Analysis

Methodology: the CSV was loaded into a SQLite table (schema in 01_schema.sql) and verified against the source file on row count, column count and data types. Annual totals and a holiday temperature comparison were then computed with SQL queries against a deduplicated, one row per hour view.

- A data quality check came before any trend analysis: **5,445 timestamps appeared two to three times each** because the weather feed sometimes logged more than one observation per hour. **traffic_volume** was confirmed identical across each duplicate group before removing duplicates, otherwise yearly totals would have been inflated.

- Annual traffic totals range from 6.8 million (2012) to 29.4 million (2017), but this is driven mostly by **how much of each year has data** (23.9% coverage in 2012 versus 99.5% in 2017), not by real demand. Comparing **average volume per recorded hour instead** removes that artefact: 2012 to 2015 sit in a narrow band (about 3,227 to 3,310 vehicles per hour), 2016 is the lowest of all six years despite good coverage, and 2017 reaches the highest average of any year, 3,376.6 per hour, which, given its near complete coverage, looks like a genuine increase in demand rather than a data artefact.
- Comparing New Year's Day and Labor Day temperatures across 2015 to 2017 found **no consistent relationship between temperature change and traffic change**: New Year's Day traffic rose alongside a 3.0-degree Celsius warming, consistent with temperature mattering, but Labor Day traffic rose 11.3% even as temperature dropped sharply between 2016 and 2017, the opposite of what a temperature driven theory would predict. **This early result already pointed toward the project's dominant later theme, that calendar and time effects outweigh weather.**

Task 2: Descriptive Statistics and Correlation

- Traffic volume statistics: **mean 3,290.65, median 3,427.00, standard deviation 1,984.77**, range 0 to 7,280. The mean sitting below the median **indicates a distribution pulled down by a cluster of near zero overnight hours**, while the standard deviation, at roughly 60% of the mean, confirms traffic volume swings widely rather than sitting near its average.
- The Pearson correlation between temperature and traffic volume across all 40,575 hours is **$r = 0.14$** (R squared under 2%), positive but very weak. **Ten rows recording an impossible 0 Kelvin and 78 recording a suspicious repeated placeholder value (276.793K) were identified and excluded** as a robustness check; the coefficient barely moved (0.1391), confirming the weak relationship is real rather than an artefact of bad readings.

This result was interpreted carefully rather than taken as evidence of causation: time of day is a clear confounder, since both traffic and temperature follow their own daily and seasonal cycles that can drift together without any causal link between them.

Task 3: Probability and Congestion Analysis

Congestion here is defined as any hour with traffic_volume above 5,500 vehicles, a fixed threshold, distinct from the data driven quartile thresholds used for congestion in Parts 2 and 3, a deliberate difference worth noting for anyone comparing figures across this report.

- **$P(\text{Congestion}) = 15.05\%$, $P(\text{Clear Weather}) = 32.94\%$, $P(\text{Congestion and Clear Weather}) = 4.34\%$** . Testing independence, $P(\text{Congestion})$ times $P(\text{Clear Weather})$ works out to 4.96%, not equal to the observed 4.34%, so congestion and clear weather are not independent.
- The **odds ratio of congestion in clear versus cloudy weather is 0.74**: congestion is about 26% less likely in clear weather than in cloudy weather. Combined with the weak correlation from Task 2, this again places weather as a minor factor next to time of day and day of week.

Task 4: Power BI Traffic Intelligence Dashboard

Methodology: Power Query was used to check row and column counts, detect that the holiday column's literal text "None" was masking missing values from the default quality view, correct temp, rain_1h and snow_1h from an auto detected Text type to Decimal Number, and derive Hour, Temperature in Celsius, and a three band Traffic Category (Low, Medium, High at the same 4,500 and 5,500 vehicle thresholds as Task 3's congestion definition).

- **Clouds has the highest average traffic volume (3,616.99)**, closely followed by Haze and Drizzle, while **Squall has the lowest by a wide margin (420.00)**, a difference of 3,197 vehicles. Squall's extremity is flagged as partly a small sample artefact rather than a robust estimate, since severe weather events are both disruptive and rare in this dataset, the same caution that resurfaces for the same two categories throughout Parts 2 and 3.
- The temperature versus traffic scatter plot shows **no visible linear relationship**, high and low volumes occur across nearly the entire temperature range, and the plot's most notable outlier cluster, points sitting near -270 degrees Celsius, was correctly identified as the same 0 Kelvin sensor error found independently in Task 2, rather than a genuine cold weather pattern.

Part 2: Python Pipeline and Analytics

Part 2 turns Part 1's manual analysis into a reproducible Python codebase: four independently runnable stages plus a shared logging system and an incrementally committed GitHub history. Full detail is in [part2_python/part2_report.docx](#); summarised here.

Task 1: Data Pipeline Construction (pipeline.py)

Methodology: the raw CSV is loaded and validated against the expected nine column schema, then cleaned in five stages, standardising categorical text, parsing and validating date_time, removing duplicates, and imputing physically impossible sensor readings using that calendar month's own median rather than one global average.

- **48,204 raw rows became 40,575 cleaned rows:** 17 exact duplicates removed and 5,430 duplicate timestamp hours (7,612 rows) collapsed to one row per hour, with zero inconsistent traffic_volume values found across any duplicate group, confirming the collapse was safe.
- 10 rows with an impossible temperature at or below 0 Kelvin and 1 row with an implausible 9,831.3 millimetre per hour rainfall reading were imputed from their calendar month's median rather than dropped, preserving the dataset's shape.

Task 2: Feature Engineering (feature_engineering.py)

- Cyclical sine and cosine encoding of hour, day of week and weekend indicators, one hot encoding of weather_main, a derived is_severe_weather flag, z score scaling of temp and traffic_volume, and a congestion_category built from the data's own 25th and 75th percentiles. Output grew from 9 to 29 columns.
- **454 of 40,575 rows (1.1%) are flagged is_severe_weather**, and the resulting congestion split, Low 10,144, Medium 20,289, High 10,142, lands close to the intended 25/50/25 design purely from the data's own quartiles.

Task 3: Visualising Traffic Patterns (visualizations.py)

- **Traffic peaks in the afternoon commute, 16:00 to 17:00 (roughly 5,300 to 5,700)**, exceeding the morning peak at 07:00 (roughly 4,770). Weekdays show a pronounced double peak and a substantially higher mean (3,557) than weekends (2,624), which show a single flatter midday hump, the clearest demonstration yet of time and calendar effects outweighing weather.
- The overall traffic_volume distribution is bimodal, reflecting quiet overnight hours and busy daytime hours as two distinct regimes rather than one smooth distribution. Haze, Clouds and Rain show the highest median volumes; Squall is a degenerate single data point, the same small sample caution raised independently in Part 1's Power BI dashboard.

Task 4: Mini Traffic Analytics Application (traffic_cli.py)

A four command CLI (query, peak-hours, compare-weekday-weekend, recommend), each rebuilding the cleaned, feature engineered dataset before answering, with malformed input caught and logged as a single clear ERROR line rather than an unhandled traceback.

- **The recommended low traffic travel windows are 05:00 (avg. 2,104) and 22:00 (avg. 2,224)**, the quiet edges of the waking day (05:00 to 22:00), deliberately avoiding a trivially quiet but impractical 3am recommendation.

Task 5: Logging, Reproducibility, and GitHub

Every module calls only logging.getLogger(__name__); a single shared helper, logging_config.py, attaches one console handler (INFO and above) and one file handler (DEBUG and above) to the root logger, invoked exactly once from whichever script's entry point actually runs. DEBUG carries internal values such as scaling parameters; INFO covers expected milestones; WARNING covers recoverable data issues handled automatically; ERROR covers problems that

stop an operation. `print()` is reserved for the CLI app's actual answer to a query, never for internal progress. All work was committed incrementally to GitHub, one descriptively named commit per task.

Part 3: Machine Learning and AI

Part 3 builds seven tasks on top of Part 2's pipeline: two families of classical machine learning, a deep learning model with explainability, experiment tracking, a recommendation system, a full MLOps simulation, and a closing responsible AI review. Its own dataset, built by `data_prep.py`, adds a documented proxy `high_risk` label since no real accident data was available.

Task 1: Supervised Machine Learning Models (`supervised_models.py`)

Methodology: a chronological, not random, 80/20 train/test split (the test set is always the most recent slice of the timeline), so no model ever trains on data from after its own test period. Two targets: the `high_risk` proxy (logistic regression and a random forest classifier) and `traffic_volume` (linear regression and a random forest regressor). Both random forests are capped at `max_depth=15`, `min_samples_leaf=10`, a constraint discussed further under Task 6.

Model	Accuracy / MAE	Precision / R2	Recall	F1 / ROC AUC
Logistic Regression (high_risk)	0.9784	0.7136	1.0000	0.8329 / 0.9978
Random Forest (high_risk)	0.9890	0.8305	1.0000	0.9074 / 0.9988
Linear Regression (traffic_volume)	MAE 828.25	R2 0.7089	-	-
Random Forest (traffic_volume)	MAE 268.78	R2 0.9411	-	-

- The random forest regressor's **MAE of 268.78 (R squared 0.9411)** comfortably outperforms the linear regression baseline (MAE 828.25, R squared 0.7089), and its feature importances are dominated by time, `hour_cos` alone carries 0.6917 of total importance, `hour_sin` a further 0.1814, together over 87%, with `temp_scaled` contributing only 0.0129, the clearest single quantitative confirmation of this report's dominant theme.
- The `high_risk` classifier's top feature is **`is_low_visibility` at 0.3720**, with `wx_Mist`, `is_severe_weather` and `wx_Thunderstorm` also ranking highly, together with `is_low_visibility` accounting for over 60% of total importance. Since these are the same weather categories the `high_risk` label is partly defined from, this is a direct, measured sign of the label circularity risk discussed fully in the bias and fairness report, not an independent discovery that weather predicts accident risk.

Task 2: Unsupervised Machine Learning (`unsupervised_models.py`)

Methodology: K means clustering on cyclical hour, weather severity and traffic volume, with k swept from 2 to 8 and compared on inertia and silhouette score before settling on k=6 (silhouette stayed in a narrow 0.33 to 0.37 band throughout, so k=6 was chosen for sitting at the inertia elbow with a near peak silhouette and an interpretable number of segments). Separately, Apriori association rule mining over discretised time of day, weather, weekday type and congestion level, filtered to rules that predict congestion.

Cluster	Rows	% of data	Avg hour	Most common weather	Avg traffic volume
0	8,617	21.2%	3.0	Clear	878
1	7,815	19.3%	15.5	Clouds	5,132
2	8,460	20.9%	9.2	Clouds	4,814

3	6,847	16.9%	20.8	Clouds	2,580
4	4,383	10.8%	12.4	Rain	4,834
5	4,453	11.0%	10.2	Rain	1,406

- The six clusters separate cleanly into recognisable regimes: cluster 0 is overnight and near empty (avg hour 3.0, avg volume 878), clusters 1 and 2 are daytime and busy (avg volume above 4,800), and clusters 4 and 5 both carry the highest average weather severity (around 3.2 to 3.3, rain dominated) but split sharply on volume, 4,834 for a midday rain cluster against 1,406 for a cooler, quieter one, showing weather's effect on volume is heavily conditional on when it occurs, not constant.
- Apriori mined **271 frequent itemsets and 1,002 rules** (minimum support 0.005, lift at least 1.0). The single strongest congestion predicting rule found was **night hours, cloudy weather, on a weekend, predicting low congestion with 90.9% confidence and a lift of 3.64**, meaning that combination is over three and a half times more likely to produce low congestion than chance alone, again a time and calendar driven pattern rather than a weather driven one.

Task 3: Deep Learning with Explainability (deep_learning_explainability.py)

Methodology: an LSTM predicts next hour traffic volume from a 24-hour lookback window. A gap analysis found that roughly 23% of all hours in the six-year window have no record at all, so every window is checked for a genuinely consecutive 24-hour span before use, rather than silently treating a gap as continuous. For explainability, SHAP was applied to Task 1's random forest regressor rather than to the LSTM directly, since SHAP's deep learning explainers are version fragile with Keras and TensorFlow, and the brief explicitly permits this substitution when the chosen model is difficult to explain directly.

- The LSTM (a single 64-unit layer, dropout, a small dense head, early stopping) reached **MAE 174.33 and R squared 0.9832** after 18 epochs, moderately more accurate than Task 1's tabular random forest on the same underlying target, consistent with sequence models capturing short term momentum that an hour-by-hour tabular model cannot see directly. Repeated runs show a small amount of run-to-run variation (roughly MAE 174 to 191, R squared 0.982 to 0.983) traced to CPU side oneDNN floating point non-determinism, documented rather than treated as a bug.
- SHAP's hour by hour time contribution (task3_hour_shap_effect.csv) shows a clean, physically sensible curve: a strongly negative contribution overnight (as low as -2,898 at 3am), turning positive from 06:00, and peaking at **+2,363 at 16:00**, matching the afternoon commute peak found independently in Part 2's visualisations and Part 1's Power BI chart, three separate methods converging on the same hour.

Task 4: MLflow Experiment Tracking (mlflow_tracking.py)

Every model trained in Tasks 1 to 3 is reloaded from disk, its exact test set rebuilt from the same functions used to train it, and its metrics recomputed fresh rather than hardcoded, then logged to a local SQLite backed MLflow instance with an explicit artefact location. This gives one place to compare every model built in the capstone side-by-side, and is the direct foundation for Task 6's registry. Its own documented limitation is that this is retrospective logging of already completed runs, not live instrumentation, and there is no automated retraining pipeline behind it.

Task 5: Traffic Recommendation System (traffic_recommendation.py)

Methodology: for each combination of day type (weekday or weekend) and weather condition, an hourly traffic profile is simulated from Task 1's random forest regressor, then searched for the best short travel window using a rush hour anchored algorithm, the maximum of the profile in the morning commute hours and the maximum in the afternoon commute hours are found first, and the genuine dip strictly between them, if one exists and is lower than both, is recommended; otherwise the result honestly reports no significant dip rather than forcing one.

This final design was reached only after three earlier approaches each failed on a different edge case, a naive lowest average search produced a search boundary artefact, a local minimum test produced a false positive on a trivial wiggle, and a prominence threshold fix produced a false negative on a genuine but flattened dip. Each failure was caught by

direct scrutiny of the output against what a plausible commuting pattern should look like, rather than accepted because the code ran without error.

- The validated system recommends **10:00 to 11:00 AM for every weekday scenario** (a genuine mid-morning trough between the two commute peaks) and **7:00 to 8:00 AM for every weekend scenario** (correctly reported as not a genuine trough, since weekend traffic has only one broad midday peak rather than two flanking ones). Smoke and Squall are documented as too rare in the data for a reliable recommendation, the same small sample caution raised independently in Parts 1 and 2.

Task 6: MLOps and Deployment Simulation

Task 6 covers five connected pieces: model versioning with a registered production model, a deployment simulation, and drift monitoring with alerting.

6.1 and 6.2: Model versioning and the MLflow Model Registry (model_versioning.py)

Version	Hyperparameters	Serialized size	Test MAE	Test R2
v1 (unconstrained)	n_estimators=300, max_depth=None, min_samples_leaf=1	882.1 MB	300.05	0.9296
v2 (milder cap)	n_estimators=200, max_depth=20, min_samples_leaf=5	92.1 MB	274.48	0.9395
v3 (production)	n_estimators=150, max_depth=15, min_samples_leaf=10	30.7 MB	268.78	0.9411

- The unconstrained version (v1) is not only 29 times larger than the production version (v3), it is also **less accurate** (MAE 300.05 versus 268.78), a direct, measured demonstration that the capped model is strictly better on both the resource and the accuracy axis at once, not a trade-off between them. v1 and v2 are logged to MLflow with full parameters and metrics but no model artefact attached, deliberately avoiding ever writing an 882MB or 92MB file into the repository; only v3 is logged with its artefact and registered under the name traffic_volume_regressor with a production alias.

6.3: Deployment simulation (deployment/app.py)

A FastAPI service loads whichever model currently holds the registry's production alias, so promoting a new version and restarting the app serves it with no code change. It accepts a raw request, a timestamp, weather condition, temperature, precipitation and cloud cover, and derives the model's 23 engineered features internally, verified beforehand against a real training row in the notebook. Every request served is appended to deployment/prediction_log.csv, the data source for the monitoring step below.

6.4 and 6.5: Drift monitoring and alerting (monitoring/drift_monitor.py)

Reads the prediction log and checks it two ways: feature drift (a mean z score for continuous fields, a share ratio for categorical ones, both against the training distribution) and prediction error drift (matched against real outcomes for any request timestamp that happens to correspond to an actual historical row). A test batch of six requests deliberately including a snow event, a thunderstorm and a holiday produced five ALERTs out of ten checks, correctly flagging the rare conditions as overrepresented and finding a real, measured **3.43 times increase in prediction error (MAE 920.79 versus a baseline of 268.78)** on that batch, evidence the model's headline accuracy figure does not hold uniformly across all conditions.

Task 7: Responsible and Sustainable AI

Task 7's full bias and fairness report, and its governance and sustainability discussion, are presented as their own document, **part3_bias_fairness_governance_report.docx**, so as not to duplicate that material here. In summary, it covers three sampling and coverage limitations (a single corridor and direction, a six year window that is now dated, and an unexplained 23% gap in hourly coverage), the specific circularity risk in the high_risk proxy label now further

confirmed by Task 1's own feature importances above, a measured example of uneven error distribution from the Task 6 monitoring run, and a governance and sustainability discussion built around this project's own MLflow registry, deployment simulation, and the concrete model size versus accuracy trade off in the table above.

Cross Part Synthesis: Themes Across the Capstone

Time and calendar effects consistently outweigh weather

Every method applied across all three parts arrives at the same conclusion independently: Part 1's SQL trends and its very weak temperature correlation ($r = 0.14$), its congestion odds ratio (0.74 for clear versus cloudy weather), Part 2's weekday versus weekend charts and its afternoon peak, and Part 3's feature importances (hour_cos and hour_sin together over 87% of the traffic volume regressor's importance) and its SHAP time contribution curve, all converge on the same hour of day and calendar driven story. This convergence across entirely different techniques, from a spreadsheet style correlation to a random forest's internal feature weights, is stronger evidence than any single method could provide alone.

A recurring discipline of catching subtle errors through skepticism, not just running code

This project's most valuable moments were not the first working version of anything, they were the direct, deliberate checks that caught a problem the first version had missed: Part 1's discovery that 88 rows carried a suspicious 0 Kelvin or 276.793 Kelvin placeholder rather than a real reading, Part 2's discovery that duplicate timestamp hours needed removing before summation, Part 3's four iteration debugging of the recommendation window algorithm, the direct question of whether a suspiciously perfect correlation was too good to be true, and the discovery and fix of a project wide logging bug that had silently merged Part 3's log output into Part 2's file since Part 3's very first script. None of these were required steps in the brief; each came from treating a clean or convenient looking result as a prompt to check further rather than as confirmation.

A resource conscious default, not an afterthought

The decision to cap every random forest's depth and leaf size (Part 3 Task 1), to keep the LSTM small (Task 3), to log historical model versions without their oversized artefacts (Task 6.1 and 6.2), and to run MLflow against a local SQLite backend rather than a hosted service, were all driven primarily by practical constraints, GitHub's file size limits and a single machine environment. The finding that the smaller, more constrained model also scored better, not merely smaller, on held out accuracy (Task 6's version table) turns this practical constraint into a genuine methodological result: restraint was not a trade off against quality at this project's scale.

A note on internal consistency

Not every definition lines up exactly across parts, and this is recorded here rather than smoothed over. Part 1 defines congestion with a fixed threshold (traffic_volume above 5,500), while Parts 2 and 3 use data driven quartile thresholds instead, appropriate choices for each task's own instructions, but a reader comparing congestion figures across this report should use the definition stated in each section rather than assuming one single threshold throughout.

Conclusion

Across three parts and sixteen numbered tasks, this capstone takes one dataset from a raw CSV file through SQL analysis, statistics, an interactive dashboard, a reproducible Python pipeline, classical and deep machine learning with explainability, a full MLOps simulation with a model registry, deployment and drift monitoring, and a closing bias, fairness, governance and sustainability review. The project's own internal consistency, the same time driven pattern surfacing from every technique, the same two rare weather categories flagged as unreliable in three independent parts, and the same discipline of catching its own errors, is offered here as evidence of a genuinely integrated body of work rather than seven to sixteen disconnected exercises submitted under one folder.

AI Assistance Disclosure

This capstone was completed with the assistance of an AI assistant (Claude), used throughout Parts 1 to 3 for drafting and refining code, debugging (including the recommendation system's window selection logic and the Part 3 wide logging bug described above), and drafting documentation and this report from the project's own real outputs, logs and

figures. Every methodology decision, every numerical result quoted in this report, and every script in this repository was reviewed, run and verified against its actual output before being accepted, including several rounds of direct correction where an initial AI suggestion was checked, found wanting against a plausible real world expectation, and revised. This disclosure is made in line with the course's own encouragement of AI assisted work.

Appendix: Where to Find Everything

This report summarises methodology and findings; the underlying, runnable evidence for every claim above lives in the GitHub repository itself. **README.md** at the repository root documents the full project structure and exact run instructions for every script in every part, and states plainly that this project uses a documented proxy risk label rather than a real accident dataset. In brief: Part 1's SQL, statistics and Power BI work is under `part1_data_analytics/`; Part 2's pipeline, feature engineering, visualisations and CLI app are under `part2_python/`; and Part 3's models, notebooks, MLflow logs, recommendation engine, and deployment and monitoring simulation are under `part3_machine_learning/`, with the bias and fairness report as `part3_bias_fairness_governance_report.docx` alongside them.